

Computational Complexity Examples

JOHAN ALENLÖV

732A94 Advanced R Programming

Department of Computer and Information Science, Linköping University

HT 2026

Examples 1–7 are based on the examples by Krzysztof Bartoszek.

Worked examples for Lecture 6, ordered as the exercise sheet. Throughout, $\log = \log_2$, n is the problem size, and arithmetic operations and comparisons cost $O(1)$.

Recap

Notation	Definition	Limit test (sufficient)
$f = O(g)$	$\exists C > 0 \exists N \forall n > N : f(n) \leq C g(n) $	$\limsup f / g < \infty$
$f = \Omega(g)$	$\exists C > 0 \exists N \forall n > N : f(n) \geq C g(n) $	$\liminf f / g > 0$
$f = \Theta(g)$	$f = O(g)$ and $f = \Omega(g)$	$ f / g \rightarrow L, 0 < L < \infty$
$f = o(g)$	$\forall C > 0 \exists N \forall n > N : f(n) \leq C g(n) $	$ f / g \rightarrow 0$

Tools:

- Change of base: $\log_a n = \log_b n / \log_b a$.
- Substitution $n = 2^m$: logarithms become polynomials, polynomials become exponentials.
- Powers of 2: if $f = 2^F$, $g = 2^G$, then $f = o(g)$ iff $F - G \rightarrow -\infty$, and $f = \Theta(g)$ iff $F - G$ is bounded.
- Sums: (number of terms) $\times$ (largest term) above, the largest half of the terms below, or compare with an integral.

Part A: Asymptotic notation

Example 1 Smallest polynomial bound

For $f(n) = (n^2 + 1)(2n^4 + 3n - 8)$, find the smallest k such that $f(n) = O(n^k)$.

Solution. $f(n) = 2n^6 + 2n^4 + 3n^3 - 8n^2 + 3n - 8$, so

$$n^{-6}f(n) = 2 + 2n^{-2} + 3n^{-3} - 8n^{-4} + 3n^{-5} - 8n^{-6} \rightarrow 2 < \infty,$$

while $n^{-5}f(n) = 2n + \dots \rightarrow \infty$. Hence $k = 6$.

Example 2 A simple bound

For what “simple” $g(n)$ does $f(n) = (4n \log n + 1)^2 \in O(g(n))$?

Solution. $f(n) = 16n^2(\log n)^2 + 8n \log n + 1$. With $g(n) = n^2(\log n)^2$:

$$\frac{f(n)}{n^2(\log n)^2} = 16 + \frac{8}{n \log n} + \frac{1}{n^2(\log n)^2} \rightarrow 16 < \infty.$$

Example 3 Logarithm of a power

True or false: $\log n^{73} = O(\log n)$?

Solution. True: $\log n^{73} / \log n = 73$. (But $(\log n)^{73} \neq O(\log n)$: the ratio is $(\log n)^{72} \rightarrow \infty$.)

Example 4 Exponential versus factorial

Show that $A^n \in O(n!)$ for $A > 0$.

Solution. Let $a = \lceil A \rceil$. For $n > a$, the factors $n, n-1, \dots, a+1$ are all $> A$, so $n! > A^{n-a}$ and

$$\frac{A^n}{n!} < \frac{A^n}{A^{n-a}} = A^a < \infty.$$

Example 5 The factorial eventually wins

Show that $A^n < n!$ for n large enough.

Solution. For $A \leq 1$ it holds for $n \geq 2$, so let $A > 1$ and $b = \lceil 2A \rceil$. For $n > b$ the factors $n, \dots, b+1$ are all $> 2A$, so

$$n! > (2A)^{n-b} = A^n 2^n (2A)^{-b}.$$

So $A^n < n!$ once $2^n \geq (2A)^b$, i.e. $n \geq b(1 + \log A)$. Take $n > \max(b, b(1 + \log A))$. Moreover $A^n/n! < (2A)^b 2^{-n} \rightarrow 0$, so $A^n = o(n!)$.

Example 6 A bounded sum

Let $s_n = \sum_{k=1}^n k^{-2}$. Show that $s_n \in O(1)$.

Solution. For $k \geq 2$, $\frac{1}{k^2} < \frac{1}{k(k-1)} = \frac{1}{k-1} - \frac{1}{k}$, and the sum telescopes:

$$s_n \leq 1 + \sum_{k=2}^n \left(\frac{1}{k-1} - \frac{1}{k} \right) = 1 + 1 - \frac{1}{n} < 2.$$

Example 7 Arithmetic sum

Let $t_n = \sum_{k=1}^n k$. Show that $t_n \in O(n^2)$.

Solution. $t_n = \frac{n(n+1)}{2}$, so $n^{-2}t_n = \frac{1}{2} + \frac{1}{2}n^{-1} \rightarrow \frac{1}{2}$. In fact $t_n = \Theta(n^2)$.

Example 8 True or false

Which of the statements below are true?

- i) $3^{n+2} = \Theta(3^n)$
- ii) $3^n = o(4^n)$
- iii) $n^{1/\log n} = \Theta(1)$
- iv) $\log(n^3 + 5n) = \Theta(\log n)$
- v) $(\ln n)^3 = o(n)$
- vi) $n \log n = o(n^2)$ and $n \log n = \Omega(n)$
- vii) $\sum_{k=1}^n \sqrt{k} = \Theta(n^{3/2})$
- viii) $\max(f(n), g(n)) = \Theta(f(n) + g(n))$ for positive f, g

Solution. All are **true**.

- i) $3^{n+2}/3^n = 9$.
- ii) $3^n/4^n = (3/4)^n \rightarrow 0$. The base of an exponential matters.
- iii) $n^{1/\log n} = 2^{\log n / \log n} = 2$.
- iv) $3 \log n \leq \log(n^3 + 5n) \leq \log 6 + 3 \log n$ for $n \geq 1$, so the ratio tends to 3.
- v) With $n = e^m$: $m^3/e^m \rightarrow 0$.
- vi) $\log n/n \rightarrow 0$, and $n \log n/n = \log n \geq 1$ for $n \geq 2$.

- vii) Above: n terms, each $\leq \sqrt{n}$, so the sum is $\leq n^{3/2}$. Below: at least $n/2$ terms have $k \geq n/2$, each $\geq \sqrt{n/2}$, so the sum is $\geq 2^{-3/2}n^{3/2}$.
- viii) $\max(f, g) \leq f + g \leq 2 \max(f, g)$.

Example 9 Ordering by growth rate

Arrange the functions below so that each one is $o(\cdot)$ of the next. Exactly one pair is Θ of each other; find it.

$$\sqrt{n}, \quad 8^{\log n}, \quad \log(n^5), \quad n^{1/\log n}, \quad 2^{\sqrt{\log n}}, \quad (2n)^3, \quad n^{\log n}, \quad 3^n$$

Solution. With $m = \log n$, write each function as $2^{F(m)}$:

f	$n^{1/\log n}$	$\log(n^5)$	$2^{\sqrt{\log n}}$	$\sqrt{n}$	$8^{\log n}$	$(2n)^3$	$n^{\log n}$	3^n
$F(m)$	1	$\log 5 + \log m$	$\sqrt{m}$	$m/2$	$3m$	$3m + 3$	m^2	$2^m \log 3$

Consecutive exponents differ by an amount tending to ∞ , except for $8^{\log n} = n^3$ and $(2n)^3 = 8n^3$, which differ by the constant 3 (ratio 8). So

$$n^{1/\log n} \prec \log(n^5) \prec 2^{\sqrt{\log n}} \prec \sqrt{n} \prec 8^{\log n} \asymp (2n)^3 \prec n^{\log n} \prec 3^n.$$

Example 10 How large a problem can we solve?

At 10^9 operations per second, what is the largest n that can be handled in one minute, if the algorithm uses $f(n)$ operations?

Solution. Solve $f(n) \leq 6 \cdot 10^{10}$:

$f(n)$	n	$n \log n$	n^2	n^3	2^n	$n!$
largest n	$6 \cdot 10^{10}$	$1.9 \cdot 10^9$	$2.4 \cdot 10^5$	3900	35	13

A $100\times$ faster computer multiplies n by 100 for $f(n) = n$, but only adds $\log 100 \approx 6.6$ for 2^n .

Part B: Loops in R

Count how often the innermost body runs (a sum over the outer variable), and check the cost of each statement: one line of R may hide a loop over a whole vector.

Example 11 A triangular double loop

```

for (i in 1:n)
  for (j in i:n)
    s <- s + 1

```

Solution. $\sum_{i=1}^n (n - i + 1) = \frac{n(n+1)}{2} = \Theta(n^2)$.

Example 12 Halving

```

i <- n
while (i > 1)
  i <- i %/% 2

```

Solution. After t iterations $i = \lfloor n/2^t \rfloor$, so the loop runs $\lfloor \log n \rfloor$ times: $\Theta(\log n)$.

Example 13 An inner loop that depends on the outer one

```

for (i in 1:n) {
  j <- 1
  while (j * j <= i)
    j <- j + 1
}

```

Solution. The inner loop runs $\lfloor \sqrt{i} \rfloor$ times, giving $\sum_{i=1}^n \lfloor \sqrt{i} \rfloor$ in total. This is $\leq n\sqrt{n}$, and $\geq \frac{n}{2}(\sqrt{n/2} - 1)$ (keep only $i \geq n/2$). Hence $\Theta(n^{3/2})$.

Example 14 Hidden loops: removing the first element

What is the complexity? Recall that `x[-1]` copies `length(x) - 1` elements.

```

drain <- function(x) {
  total <- 0
  while (length(x) > 0) {
    total <- total + x[1]
    x <- x[-1]
  }
  total
}

```

Solution. The loop runs n times, but at length m the copy costs $m-1$: $\sum_{m=1}^n (m-1) = \frac{n(n-1)}{2} = \Theta(n^2)$. Using an index, or `sum(x)`, gives $\Theta(n)$.

Part C: Exact counts, sorting and timings

Example 15 Closest pair of numbers

Find the smallest $|x_i - x_j|$, $i \neq j$, for a vector x of length $n \geq 2$.

```

closest_naive <- function(x) {
  n <- length(x)
  best <- Inf
  for (i in 1:(n - 1))
    for (j in (i + 1):n)
      best <- min(best, abs(x[i] - x[j]))
  best
}

closest_sort <- function(x) {
  y <- sort(x)
  min(diff(y)) # diff(y) is y[2] - y[1], ..., y[n] - y[n - 1]
}

```

- How many distances does `closest_naive` compute?
- Show that `closest_sort` is correct, and give its complexity.

Solution. a) $\sum_{i=1}^{n-1} (n-i) = \frac{n(n-1)}{2} = \Theta(n^2)$.

b) For $p < q$ in the sorted vector, $y_q - y_p = (y_q - y_{q-1}) + \dots + (y_{p+1} - y_p) \geq y_{p+1} - y_p$, since every term is ≥ 0 . So the minimum is attained by neighbours, i.e. by `diff(y)`. The cost is $O(n \log n)$ for sorting plus $\Theta(n)$ for the rest: $O(n \log n)$.

Example 16 Reading complexity off a timing table

Running times (seconds) of two implementations:

n	1000	2000	4000	8000	16000
A	0.002	0.008	0.033	0.13	0.51
B	0.011	0.020	0.041	0.080	0.16

Estimate k in $T(n) \approx Cn^k$, predict the time for $n = 64000$, and find where the curves cross.

Solution. When n doubles, $k \approx \log(T(2n)/T(n))$. A's time is multiplied by about 4 at each doubling, so $k_A \approx 2$. B's is multiplied by about 2 (the first ratio, 1.8, is lowered by fixed overhead), so $k_B \approx 1$.

Prediction: $T_A(64000) \approx 4^2 \cdot 0.51 \approx 8.2$ s, and $T_B(64000) \approx 4 \cdot 0.16 = 0.64$ s.

Crossover: $C_A \approx 0.51/16000^2 \approx 2 \cdot 10^{-9}$ and $C_B \approx 0.16/16000 = 10^{-5}$. The curves cross where $C_A n^2 = C_B n$, i.e. $n = C_B/C_A \approx 5000$. Below that, A's much smaller constant wins.

Part D: Recursion and the master theorem

d is *multiplicative* if $d(xy) = d(x)d(y)$; for example n^p and 1, but not cn with $c \neq 1$, nor $n \log n$. For multiplicative d and $n = b^k$:

$$T(n) = aT(n/b) + d(n), \quad T(1) = \Theta(1) \quad \implies \quad T(n) = \begin{cases} \Theta(n^{\log_b d(b)}) & a < d(b), \\ \Theta(n^{\log_b a} \log n) & a = d(b), \\ \Theta(n^{\log_b a}) & a > d(b). \end{cases}$$

For $d(n) = cn^p$, write $T = c\tilde{T}$ as in the lecture.

Example 17 Applying the master theorem

Solve the recurrences (n a power of b , $T(1) = \Theta(1)$).

- $T(n) = 2T(n/2) + 1$
- $T(n) = 2T(n/2) + n^2$
- $T(n) = 9T(n/3) + n^2$
- $T(n) = 8T(n/2) + n^2$ and $T(n) = 7T(n/2) + n^2$
- $T(n) = 2T(n/2) + n \log n$, $T(1) = 0$

Solution.

- $d(2) = 1 < 2 = a$: $\Theta(n^{\log_2 2}) = \Theta(n)$.
- $d(2) = 4 > 2$: $\Theta(n^{\log_2 4}) = \Theta(n^2)$.
- $d(3) = 9 = a$: $\Theta(n^{\log_3 9} \log n) = \Theta(n^2 \log n)$.
- $d(2) = 4$. For $a = 8$: $\Theta(n^3)$ (block matrix multiplication). For $a = 7$: $\Theta(n^{\log_2 7}) \approx \Theta(n^{2.81})$ (Strassen).
- $n \log n$ is not multiplicative, so unroll with $n = 2^k$:

$$T(n) = \sum_{j=0}^{k-1} 2^j \frac{n}{2^j} \log \frac{n}{2^j} = n \sum_{j=0}^{k-1} (k-j) = n \frac{k(k+1)}{2} = \Theta(n(\log n)^2).$$

Example 18 Copies in a recursive function

Give and solve the recurrences ($n = 2^k$). Recall that `x[1:m]` copies m elements.

```
sum_copy <- function(x) {
  n <- length(x)
  if (n == 1) return(x[1])
  m <- n %/% 2
  sum_copy(x[1:m]) + sum_copy(x[(m + 1):n])
}

sum_index <- function(x, lo = 1, hi = length(x)) {
  if (lo == hi) return(x[lo])
  m <- (lo + hi) %/% 2
  sum_index(x, lo, m) + sum_index(x, m + 1, hi)
}
```

Solution. **sum_copy**: the two halves copy n elements in total, so $T(n) = 2T(n/2) + cn$. Here $a = 2 = d(2)$, giving $\Theta(n \log n)$.

sum_index: no copying, so $T(n) = 2T(n/2) + c$, giving $\Theta(n)$ by Example 17a.

Example 19 When the master theorem does not apply: $T(n-1)$

```
rec_max <- function(x) {
  if (length(x) == 1) return(x[1])
  max(x[1], rec_max(x[-1]))
}
```

Solution. The copy $x[-1]$ costs $n-1$, so $T(n) = T(n-1) + c(n-1) + c_1$. The size shrinks by subtraction, not division, so unroll:

$$T(n) = T(1) + \sum_{m=2}^n (c(m-1) + c_1) = \Theta(n^2).$$

Example 20 Counting calls and proof by induction

```
g <- function(n) if (n == 0) 1 else g(n-1) + g(n-1)
h <- function(n) if (n == 0) 1 else 2 * h(n-1)
```

Both return 2^n . Let $C(n)$ be the number of calls made by $g(n)$, including the first. Find $C(n)$, prove it by induction, and compare with h .

Solution. $C(0) = 1$ and $C(n) = 1 + 2C(n-1)$. The values 1, 3, 7, 15, ... suggest $C(n) = 2^{n+1} - 1$.

Induction. Base case: $C(0) = 1 = 2^1 - 1$. Induction step: if $C(n-1) = 2^n - 1$, then $C(n) = 1 + 2(2^n - 1) = 2^{n+1} - 1$. $\square$

So g is $\Theta(2^n)$. h makes one recursive call per level, so $n+1$ calls in total: $\Theta(n)$. g recomputes the same subproblem twice; reusing the value, or memoising it, removes the blow-up.